
LLD – DAY 4

UML Diagrams (Unified Modeling Language)

◇ What is UML?

UML is a **standard visual language** used to:

- Design systems
- Represent structure & behavior
- Communicate design clearly before coding

✂ UML is heavily used in **LLD rounds**.

◇ Types of UML Diagrams

☑ Two Main Categories

1 Structural Diagrams (Static)

☞ Show **structure of the system**

Examples:

- Class Diagram
 - Object Diagram
 - Component Diagram
-

2 Behavioral Diagrams (Dynamic)

☞ Show **interaction / behavior over time**

Examples:

- Sequence Diagram
- Activity Diagram
- State Diagram

✂ **Interview Tip**

Structural = *What exists* Behavioral = *How it behaves*

◇ 1. CLASS DIAGRAM (Most Important)

◇ What is a Class Diagram?

A **Class Diagram** represents:

- Class name
- Variables (attributes)
- Methods (behaviors)
- Relationship between classes

◇ A.Class Structure in UML

A class is divided into **3 parts**:

Class Name

Variables

Methods

◇ Example: Car Class

🧑‍💻 Given Code

```
class Car {
private:
    string brand;
    string model;
    int engineCC;

public:
    startEngine();
    stopEngine();
    accelerate();
    brake();
};
```

✍ UML Representation

Car

- brand : string
- model : string
- engineCC : int

```

-----
| + startEngine() : void |
| + stopEngine() : void |
| + accelerate() : void |
| + brake() : void      |
-----

```

◇ Access Modifiers in UML

Access	Symbol	Meaning
public	+	Anywhere
protected	#	Class + child
private	-	Same class only

🧠 Visibility Table

Scope	public	protected	private
Within class	✓	✓	✓
Child class	✓	✓	✗
Outside class	✓	✗	✗

◇ B.Association (UML – Class Relationship)

◇ Association ke MAIN 2 Types hote hain

```

Association
├── Class Association
│   └── Inheritance
└── Object Association
    ├── Simple Association
    ├── Aggregation
    └── Composition

```

◇ 1. Class Association

☑ Definition

When **one class is related to another class at design level**, it is called **Class Association**.

📌 Class association ka main example = Inheritance

◇ Inheritance (IS-A relationship)

☑ Definition

Child class **IS-A** type of parent class.

🧠 Example

- ManualCar **IS-A** Car
- ElectricCar **IS-A** Car

✍ UML Representation

- Hollow triangle arrow

```
ManualCar -----▷ Car
```

📌 Key Points

- Compile-time relationship
 - Reusability
 - Strong hierarchy
-

◇ 2. Object Association

☑ Definition

When **objects of two classes are related at runtime**, it is called **Object Association**.

📌 Object Association ko hi **HAS-A relationship** bhi kehte hain.

◇ Object Association ke 3 Types hote hain

◇ a. Simple Association

☑ Definition

One object **knows or uses another object**, but **both are independent**.

🧠 Example

- Person has a House
- Teacher teaches Student

✎ Dono objects **independently exist** kar sakte hain.

✎ UML Representation

Person ----- House

✎ Key Points

- Weak relationship
 - No ownership
 - Lifetime independent
-

◇ b. Aggregation (Weak HAS-A)

☑ Definition

Child object **can exist independently** of parent object.

🧠 Example

- Room has Sofa, Chair, Bed
 - Sofa can exist without Room
-

✎ UML Representation

- **Hollow Diamond**

Room ◇----- Sofa

✎ Key Points

- Whole-part relationship
 - Weak ownership
 - Child ka lifecycle parent se independent
-

◇ c. Composition (Strong HAS-A)

☑ Definition

Child object **cannot exist without parent object**.

Example

- Car has Engine
- Engine cannot exist without Car

UML Representation

- **Filled Diamond**

Car ◆----- Engine

◇ Composition – Code Example

```
#include <iostream>
using namespace std;

// Part class
class Engine {
public:
    void start() {
        cout << "Engine started" << endl;
    }
};

// Whole class (Composition)
class Car {
private:
    Engine engine;    // Strong HAS-A relationship

public:
    void startCar() {
        engine.start();
        cout << "Car is running" << endl;
    }
};

int main() {
    Car car;
    car.startCar();
    return 0;
}
```

Key Point

Engine lifecycle depends on Car → **Composition**

◇ 2. Sequence Diagram (UML – Behavioral Diagram)

☑ Definition

A **Sequence Diagram** is a **behavioral (dynamic) UML diagram** that shows **how objects interact with each other over time** by exchanging messages in a specific sequence.

✎ It focuses on:

- **Order of execution**
- **Interaction between objects**
- **Flow of control**

◇ Components of Sequence Diagram

1. Actor

External entity that initiates the interaction.

✎ Example: **User**

2. Object

Participating entities involved in interaction.

✎ Example:

- ATM
- Transaction
- Account
- CashDispenser

3. Lifeline

Vertical dashed line representing **object's lifetime during interaction**.

```
Object
|
|
|
```

4. Messages

Communication between objects.

◇ a) Synchronous Message

- Caller **waits** for response
- Solid arrow

```
ATM -> Account : verifyPIN()
```

◇ b) Asynchronous Message

- Caller **does not wait**
- Open arrow

```
Transaction -> CashDispenser : dispenseCash()
```

5. Create / Destroy Message

◇ Create

- Object creation

```
ATM -> Transaction : create()
```

◇ Destroy

- Object termination (✕ mark)

```
Transaction ✕
```

6. Lost & Found Messages

◇ Lost Message

A **lost message** is a message that is **sent but its receiver is unknown or not shown** in the diagram.

✎ Used when:

- Receiver is outside system
- Receiver is not modeled

✎ Notation

- Arrow goes to a **filled black circle**

```
ATM -> • : logTransaction()
```

🔗 Example:

- ATM sends log data to an unknown logging system
-

◇ Found Message

A **found message** is a message that **originates from an unknown sender** and is received by a known object.

🔗 Used when:

- Sender is external / unknown
- Interaction starts outside system boundary

✎ Notation

- Arrow comes from a **filled black circle**

```
• -> ATM : systemRestart()
```

7. Condition (**alt**)

Used for **if-else** logic.

```
alt [PIN valid]
  ATM -> Transaction : proceed()
else [PIN invalid]
  ATM -> User : showError()
```

8. Loop (**loop**)

Used for **repetition (for / while)**.

```
loop [retry PIN < 3]
  User -> ATM : enterPIN()
```

◇ ATM SEQUENCE DIAGRAM (Withdrawal Flow)

🗯 Objects Involved

- **User** (Actor)
 - **ATM**
 - **Transaction**
 - **Account**
 - **CashDispenser**
-

📋 ATM Withdrawal Flow (Step-by-Step)

1. User inserts card
 2. User enters PIN
 3. ATM verifies PIN
 4. Transaction object is created
 5. Account balance is checked
 6. Cash is dispensed
 7. Amount is returned to user
-

✍ TEXTUAL UML SEQUENCE DIAGRAM (EXAM-FRIENDLY)

```
User -> ATM : withdraw(amount, accNo)
ATM -> Transaction : createTransaction()
Transaction -> Account : checkAmount(amount)
Account -> Transaction : return true
Transaction -> CashDispenser : dispenseCash(amount)
CashDispenser -> ATM : cashDispensed(amount)
ATM -> User : returnAmount(amount)
```

◇ ATM Sequence Diagram with Condition & Loop

```
User -> ATM : insertCard()
User -> ATM : enterPIN()

alt [PIN valid]
    ATM -> Account : verifyPIN()
    ATM -> Transaction : createTransaction()
    Transaction -> Account : checkBalance(amount)
    Account -> Transaction : OK
    Transaction -> CashDispenser : dispenseCash()
    ATM -> User : collectCash()
else [PIN invalid]
```

```
    ATM -> User : showError()  
end  
  
loop [retry PIN < 3]  
    User -> ATM : enterPIN()  
end
```

Quick Revision (Exam / Interview)

- ✓ Sequence diagram = **Behavioral UML**
- ✓ Shows **interaction + time order**
- ✓ Uses **lifelines & messages**
- ✓ **alt** = if/else
- ✓ **loop** = repetition

DAY 4 SUMMARY (Revision Sheet)

- ✓ UML = design language
- ✓ Structural = static
- ✓ Behavioral = dynamic
- ✓ Class Diagram = structure
- ✓ Sequence Diagram = interaction
- ✓ Inheritance = IS-A
- ✓ Composition = strong HAS-A
- ✓ Aggregation = weak HAS-A

Interview Golden Lines

- **Class diagram represents system structure**
 - **Sequence diagram shows object interaction over time**
 - **Composition is stronger than aggregation**
 - **UML is language independent**
-